

Robotics Security Platform

Interview Preparation — Part 2 of 4: The Plant Floor (OT & Safety)

This part goes down into the most important and most dangerous zone: the OT plant floor. We cover the PLC and the code it runs, the robot it drives, the Modbus language they speak (and how it is attacked), the SROS2 certificate security on the robot's messaging, and — the centrepiece — the independent safety system that guarantees the robot fails safe. By the end you can explain exactly how a stopped robot stays stopped until a human says otherwise.

WHY IT MATTERS

The single most important sentence in this whole part: **"Safety is independent of security, and clearing a safe state is always a deliberate human action."** Everything below is evidence for that sentence.

1. The PLC and its program

1.1 What the PLC is doing

The production PLC runs **OpenPLC**, and its program is the file **vm-ot/openplc/production.st**, written in **Structured Text** (an industrial programming language, standard IEC 61131-3). The program is a **6-step pick-and-place cycle** that repeats on a fixed **100 ms scan** — meaning the whole program re-runs ten times a second, forever.

```
step 0: idle      -> wait for a start button
step 1: convey    -> run conveyor, enable arm      (1.0 s)
step 2: move      -> arm moves to the pick position (1.5 s)
step 3: grip      -> close the gripper              (0.5 s)
step 4: move      -> arm moves to the place position (1.5 s)
step 5: release   -> open the gripper               (0.5 s)
step 6: complete  -> count one finished cycle, loop back to step 1
```

It exposes its state on Modbus registers so the rest of the system can watch it: **%MW0** = current step, **%MW1** = completed-cycle count, **%MW2** = how many times the E-stop has tripped.

1.2 The safety-first interlock (the most important lines of PLC code)

Right at the top of the program, before any motion logic, is the safety interlock. In plain words it says: **"If the emergency stop is active OR a safe-state has been requested, immediately drop every output (motor, gripper, conveyor) and hold at step 0."**

```
safe_state_now := e_stop_active OR request_safe_state ;

IF safe_state_now THEN
    motor_arm_enable := FALSE ;    -- kill the arm
    gripper_close    := FALSE ;    -- release
    conveyor_run     := FALSE ;    -- stop the belt
    cycle_step       := 0 ;        -- freeze at the start
    (count one E-stop trip the first scan it goes active)
END_IF ;
```

WHY IT MATTERS

Because this check runs first on every 100 ms scan, there is no situation where the machine keeps moving once a safe-state is asserted. That is what "fail safe" looks like in actual code — not a comment, an interlock at the top of the loop.

There is also a **slow mode**: a flag (%MW4) that, when set, roughly doubles every step's duration — the software equivalent of slowing the line to a reduced safe speed instead of fully stopping it. The incident-response engine uses this (Part 3).

2. The robot (the simulated arm)

The physical arm is simulated in **Gazebo** (a 3D robotics simulator) and driven by a ROS2 program, **vm-ot/gazebo/cyclic_motion.py**. It is a 6-jointed arm (joints j1–j6) that follows a fixed, recorded trajectory between five waypoints:

- A = home pose → B = above the pick zone → C = at the pick zone → D = above the drop zone → E = at the drop zone.
- Motion is smoothed with cosine interpolation, so the movement looks natural — and so the ML models see coherent, learnable dynamics.

Crucially, the motion node **subscribes to the safety state** (the `/safety/state` topic). When the safety system says EMERGENCY, the robot stops commanding new positions and **freezes at its last waypoint**. So the robot itself obeys the safety system, not just the PLC.

PLAIN-ENGLISH EXAMPLE

Why simulate the robot? Because you **never test attacks on a real production line** — a false move can injure someone or cost a fortune in downtime. The Gazebo arm is a "digital twin": a realistic copy you can safely attack and experiment on. Real plants do exactly this. (You will use this point again when defending the design in Part 4.)

3. Modbus, and how it gets attacked

3.1 How Modbus works (in one minute)

Modbus/TCP is how the PLC is read and commanded over the network. It has two kinds of data: **coils** (single on/off bits, like "conveyor on/off") and **registers** (16-bit numbers, like "current step = 3"). Every operation is a numbered **function code** — for example FC 3 reads registers, FC 6 writes one register, FC 16 writes many. **Reads are FC 1–4; writes are FC 5, 6, 15, 16, 22, 23.**

WHY IT MATTERS

Modbus has **no authentication, no encryption, no integrity check**. Whoever can reach the port can command the PLC. This is not a bug we introduced — it is the reality of most industrial protocols, and it is precisely *why* the rest of this platform (segmentation, monitoring, the independent safety system) has to exist.

3.2 The four traffic generators

To have something to detect, the project ships one "normal" generator and three attack simulators (in **vm-ot/traffic/**). The normal one teaches the ML models what good looks like; the attacks deviate from it on purpose.

Script	What it does	What is supposed to catch it
modbus_normal.py	Acts as an authorised HMI: reads a fixed set of registers at a steady 5 Hz. This is the baseline of "normal".	Nothing — it is the good traffic the models learn from.
attack_modbus_inject.py	Command injection: writes to coils the baseline never writes (motor enable, conveyor) and forces odd step values.	Suricata rule, the ML models (write from a non-HMI source IP), and the safety supervisor independently.
attack_modbus_flood.py	Flood: a high-rate burst of writes — the volume itself is the attack.	The ML models (message-rate and write-ratio features spike).
attack_modbus_replay.py	Replay: re-sends previously captured legitimate-looking frames to fake activity.	The ML models and the safety supervisor's replay/regression guard.

SAY THIS IN THE INTERVIEW

"The classic cyber-physical attack is a Modbus write-burst from outside the OT zone — a register that is normally only read suddenly gets written, from an IP that is not the HMI. My feature pipeline is built around exactly that signal (see the `n_external_writes` feature in Part 3)."

4. SROS2 — securing the robot's messaging

The robot's own messages (joint commands, and the all-important safety topics) travel over ROS2/DDS. By default DDS is open — any program on the network can publish or subscribe. **SROS2** closes that with a certificate system (a small **PKI**, public-key infrastructure).

4.1 The keystore and enclaves

A bootstrap script (**bootstrap_keystore.sh**) creates a certificate authority and four "enclaves" — each enclave is an identity with its own certificate:

Enclave (identity)	Role
/lab/safety_supervisor	Owns the safety topics: publishes /safety/state, subscribes to /safety/request.
/lab/production_plc	The heartbeat publisher; allowed to publish E-stop requests on /safety/request.
/lab/ai_subscriber	Read-only: may subscribe to /safety/state (the analytics tier just watches).
/lab/cyclic_motion	The robot motion node; subscribes to /safety/state so it can freeze.

There is also a deliberately **absent** identity, /lab/test_unauthorized, used by a test to prove that a program *without* a valid certificate is rejected and cannot speak the protocol at all.

4.2 The honest, important nuance: authentication vs authorization

This is a favourite interview probe, so be precise:

- **Authentication IS enforced.** Every participant must present a certificate signed by the lab's authority. An attacker with no certificate cannot join the robot's DDS conversation — full stop.
- **Topic-level authorization (ACLs) is NOT enforced** in the running system. The intended rules (e.g. "ai_subscriber must never publish to /safety/request") exist as XML files but are not active.

TRADE-OFF / BE HONEST

Be honest about why: when the authors signed and installed the per-topic permission files, **Cyclone DDS silently hung during discovery** — the whole robot messaging layer failed to start. They reverted to the default (authenticate, but do not enforce per-topic ACLs) to keep the system working, and documented the ACL files as intended future work. "Authentication is enforced cryptographically; topic ACLs are the documented next step once the DDS discovery issue is resolved." Saying this scores maturity points.

5. The safety system (the heart of the OT story)

This is the part to know cold. The safety function is split across **three small programs** so that the guardian is independent of the thing it guards.

5.1 The three players

Program	Where it runs	Its one job
safety_heartbeat.py	Production side	Every 0.2 s (5 Hz) write a "I am alive" counter to the safety controller over Modbus.
safety_supervisor.py	The safety controller on port 503	The real "safety brain": watchdog, latch, and replay guard. Decides NORMAL / DEGRADED / EMERGENCY.
safety_bridge.py	The single SROS2 node	Translate between the secure robot topics and Modbus, and mirror the safety state onto the production PLC and the robot.

A GOOD STORY TO TELL

A bit of project history worth knowing: a simpler stand-in used to guard port 503 (no watchdog, no latch) while the documentation described the smart supervisor. That gap was closed — **safety_supervisor.py now actually runs the port-503 server in "--modbus-only" mode**, so the real watchdog/latch/replay logic is genuinely on duty. If asked about "claimed vs actual", this is the honest answer: it was a gap, and it was fixed.

5.2 The register contract (how they talk over Modbus port 503)

Production heartbeat WRITES:	Supervisor MIRRORS OUT (read-only):
HR[0] hb_counter (counts up, wraps)	HR[10] safety_state 0=NORMAL
HR[1] prod_state (1 = running)	1=DEGRADED
HR[2] remote_estop 0 = normal	2=EMERGENCY
1 = E-stop request	HR[11] ack_counter (echoes hb)
9 = admin RESET	HR[12] last_fault (why it tripped)

5.3 The four ways the supervisor trips EMERGENCY

- **Heartbeat lost (watchdog).** If no fresh heartbeat arrives for ~2 seconds, the production side is presumed dead or cut off — trip EMERGENCY. (The watchdog only arms after the first heartbeat, so startup is not a false trip.)
- **Production reports a fault.** If the production side sets prod_state = fault, trip.
- **Remote E-stop.** If an E-stop request arrives (HR[2]=1 via Modbus, or an authenticated request over SROS2), trip.
- **Replay / regression.** If the heartbeat counter steps *backwards* a little, that smells like replayed old traffic — trip with FAULT_REPLAY. (A *big* backwards jump is treated as an honest restart or 16-bit wrap, not an attack, so a reconnect does not cause a false alarm.)

5.4 Latched — and only a human clears it

Once EMERGENCY is set, it is **latched**: the supervisor will not return to NORMAL on its own, even if the heartbeat comes back. The **only** way out is a deliberate reset — writing the value **9** to register HR[2]. That mirrors real functional-safety practice: a machine must never silently restart itself after a safety event; a person must look, decide, and reset.

SAY THIS IN THE INTERVIEW

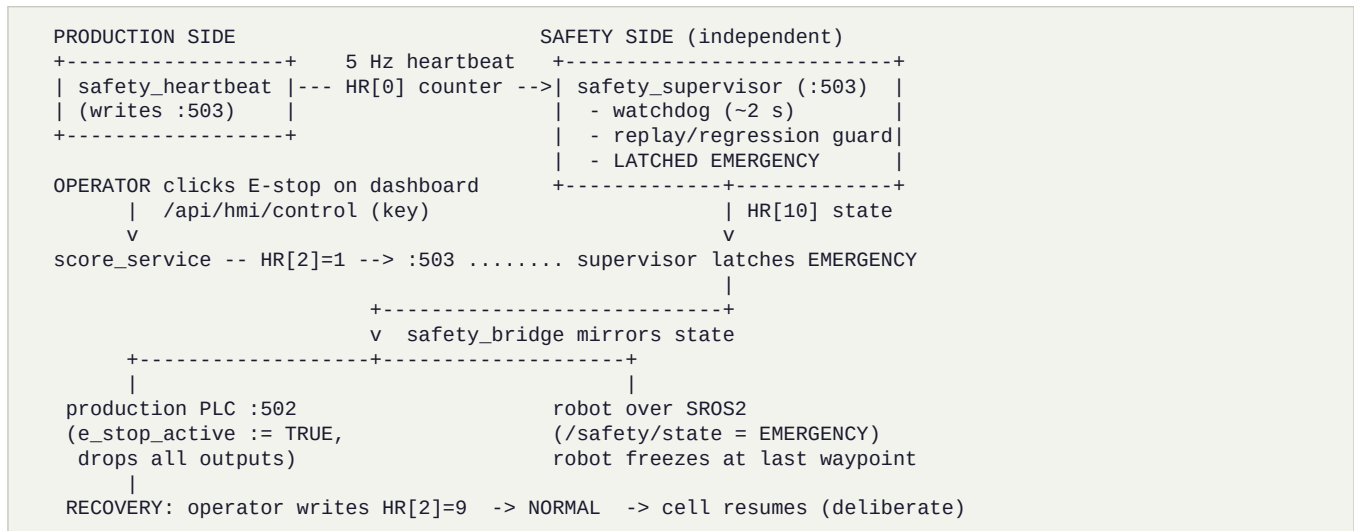
"EMERGENCY is latched. Loss of heartbeat trips it within about two seconds, and it stays tripped until a deliberate reset — the robot cannot un-stop itself." Then add the honest caveat below.

TRADE-OFF / BE HONEST

Honest caveat: the reset (register = 9) exists as a Modbus write. In the IDMZ, the analytics zone and IT have **no firewall path to the safety port** — any legitimate control rides the authenticated OT control gateway, not a raw write — so it is not network-reachable from outside OT. The remaining gap is that the safety controller is still a **software simulator co-located in OT**. **For a real cell, the top change is an independent hardware SIS (IEC 61511) whose reset is a local, physical, deliberate action with no network path at all.**

6. The complete safety & control loop

Putting it together — this is the picture to draw on a whiteboard:



Likely questions (with crisp answers)

Question	Answer
What if the network drops?	The heartbeat stops, the watchdog trips EMERGENCY within ~2 seconds, and the robot freezes. The loop is local, so a WAN/internet outage never affects it.
Could an attacker un-latch the E-stop?	In the lab the reset is over Modbus (demo convenience), but the ports are bound to localhost. In production I would remove every network path to the reset and make it physical-only.
Is the safety system really independent?	It runs as its own process on its own port with its own watchdog and latch, separate from the production controller. In a real plant I would make it a physically separate Safety Instrumented System (IEC 61511).
Authentication or authorization on ROS2?	Authentication is enforced (every node needs a CA-signed certificate). Topic-level ACLs are at the permissive default because enforcing them hung Cyclone DDS; that is documented future work.

End of Part 2. You can now explain the plant floor: the PLC cycle and its safety interlock, the simulated robot, the Modbus protocol and its attacks, SROS2 certificate security, and the independent, latched safety system. **Part 3** goes up into the brain: how traffic becomes features, how three ML models score it, how incidents are handled, and how the code itself is secured by the CI/CD pipeline.